

Lab 4 - Control Unit (CU)

CEG2136 Section A02

Teaching Assistant: Jay Kshirsagar

Date of Lab: November 12, 2025

Submission Date: December 6, 2025

Lab Performed By:

Mohammadjavad Aghaeipour Kalyani (300427782)

Sai Yarlagadda (300283438)

Lab Theory

Introduction

In this lab, we worked on creating, implementing, and testing the Control Unit (CU) for a simple RISC computer using Quartus II. The CU manages how instructions are carried out, using a set series of small steps controlled by a counter. We made the CPU work completely by combining the parts we built before with the Instruction Decoder and the timing stuff needed to run through a full instruction cycle.

Discussion of the Problem

The lab instructions provided the block diagram of the simplified computer architecture (Figure 1), including the CPU datapath, sequence counter, and memory. The problem consisted of designing and implementing the Control Unit that drives the Display, Fetch, and Execute sequence according to the instruction set (Figure 2). The Control Unit is responsible for generating the correct signals based on the opcode, operand, and timing signals.

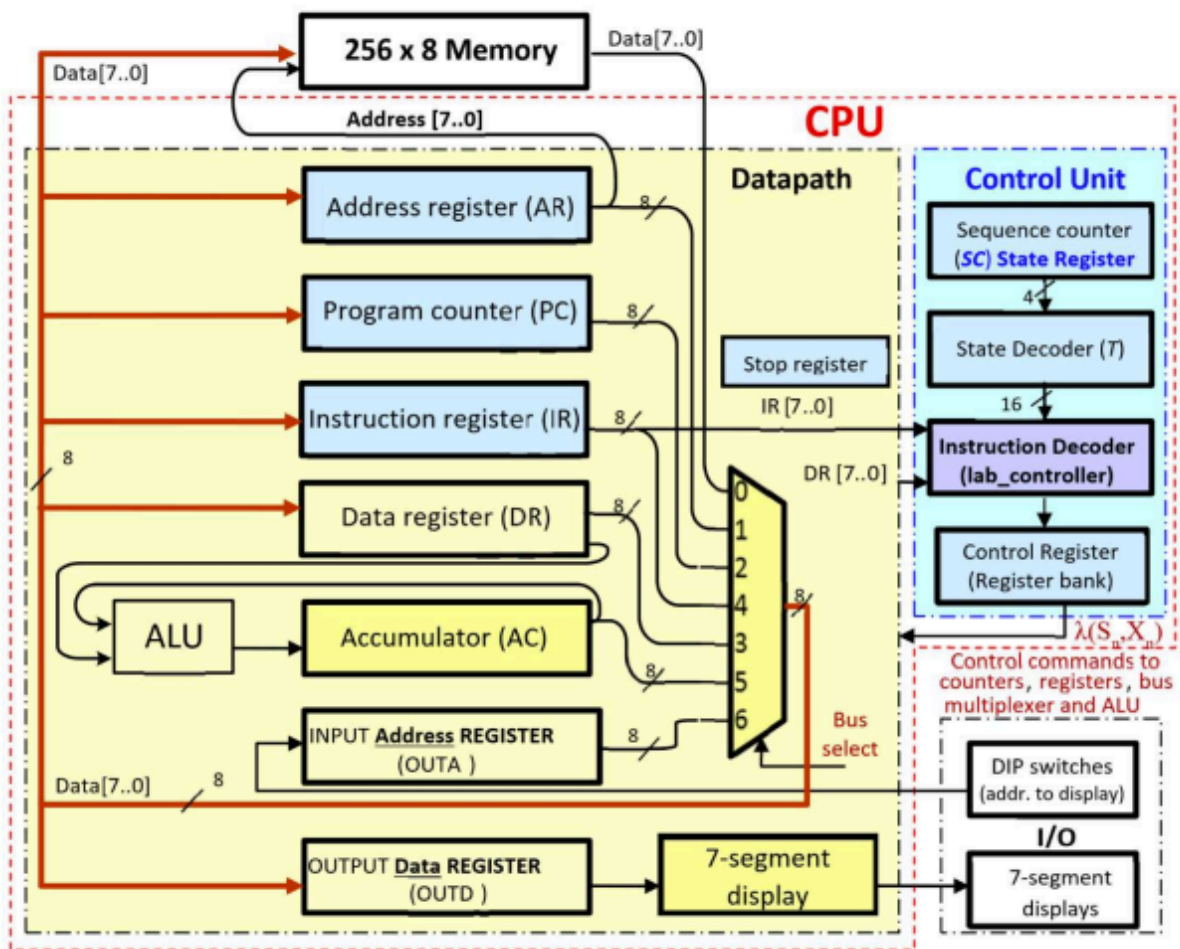


Figure 1: Computer Block Diagram

Type of Instruction	Symbol	Binary/hex opcodes		Description
		Direct Addressing I = IR6 = 0	Indirect Addressing I = IR6 = 1	
	LDD	00000000=0x00	01000000=0x40	Load AC directly from instruction
Memory Reference (IR7=0)	AND	00000001=0x01	01000001=0x41	AND AC to memory word
	ADD	00000010=0x02	01000010=0x42	Add AC to memory word
	SUB	00000011=0x03	01000011=0x43	Subtract a memory word from AC
	LDA	00000100=0x04	01000100=0x44	Load AC from a memory word
	STA	00001000=0x08	01001000=0x48	Store AC to a memory word
	BUN	00010000=0x10	01010000=0x50	Branch unconditionally
	ISZ	00100000=0x20	01100000=0x60	Increment memory word and skip the following instruction if the result is zero
Register Reference (IR7=1)	CLA	10000001=0x81		Clear AC
	CMA	10000010=0x82		Complement AC
	ASL	10000100=0x84		Arithmetic shift left AC
	ASR	10001000=0x88		Arithmetic shift right AC
	INC	10010000=0x90		Increment AC
	HLT	10100000=0xA0		Halt

Figure 2: CPU Instruction Set

Discussion of the Algorithmic Solution

Based on the micro-operation and control signal tables, the provided hardware equations (Figure 3) were implemented using logic gates in Quartus. These equations determine the register load signals, bus selection signals, and ALU control lines at each time state. The logic was added into the lab4top schematic to complete the Control Unit design.

$$\text{mem} = T_9 Y_4 + T_{10} Y_6$$

$$\text{AR_LOAD} = T_0 T_2 + T_5 IR'_7 + T_6 IR'_7 + T_7 X_2$$

$$\text{PC_LOAD} = T_8 Y_5$$

$$\text{PC_INC} = T_2 S' + T_5 IR'_7 S' + \text{DR_zero} S' Y_6 (T_{11} + T_{12})$$

$$\text{DR_LOAD} = T_8 (Y_0 + Y_1 + Y_2 + Y_3 + Y_6)$$

$$\text{DR_INC} = T_9 Y_6$$

$$\text{IR_LOAD} = T_3$$

$$\times \text{AC_CLEAR} = T_5 X_2 IR_0$$

$$\times \text{AC_LOAD} = T_5 X_2 (IR_1 + IR_2 + IR_3) + T_9 (Y_0 + Y_1 + Y_2 + Y_3)$$

$$\times \text{AC_INC} = T_5 X_2 IR_4$$

$$\text{OUTD_LOAD} = T_1$$

$$\times \text{ALU_SEL2} = T_5 X_2 IR_1 + T_9 Y_3 + T_9 Y_0$$

$$\text{ALU_SEL1} = T_9 Y_3 + T_5 X_2 (IR_1 + IR_2 + IR_3)$$

$$| \text{ALU_SEL0} = T_9 Y_2 + T_5 X_2 (IR_3 + IR_1)$$

$$\text{BUS_SEL2} = T_0 + T_9 Y_4$$

$$\text{BUS_SEL1} = T_2 + T_5 IR'_7 + T_{10} Y_6 + T_0$$

$$\text{BUS_SEL0} = T_8 Y_5 + T_{10} Y_6 + T_9 Y_4$$

$$\times \text{SC_CLEAR} = T_5 X_2 + T_9 (Y_0 + Y_1 + Y_2 + Y_3 + Y_4) + T_8 Y_5 + T_{12} Y_6$$

$$\text{HALT} = T_5 X_2 IR_5$$

Figure 3: Hardware Equations

Lab Design

Logic Diagrams and Design

Based on the equations derived from Figure 3, we were able to make the complete lab4top circuit as follows:

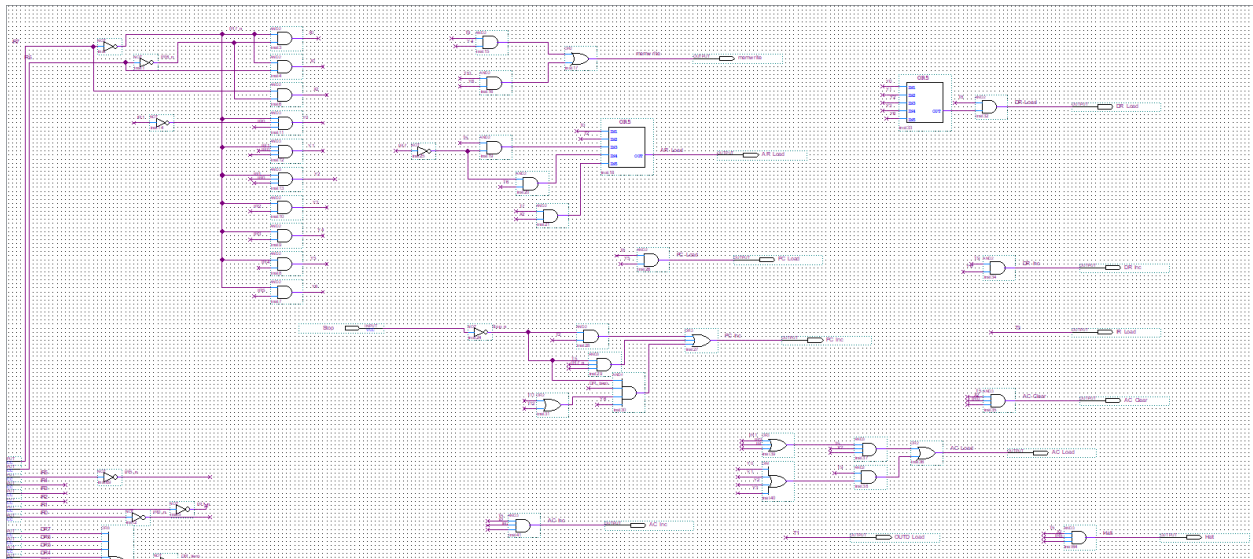


Figure 4: First Half of Completed Circuit Diagram of lab4top

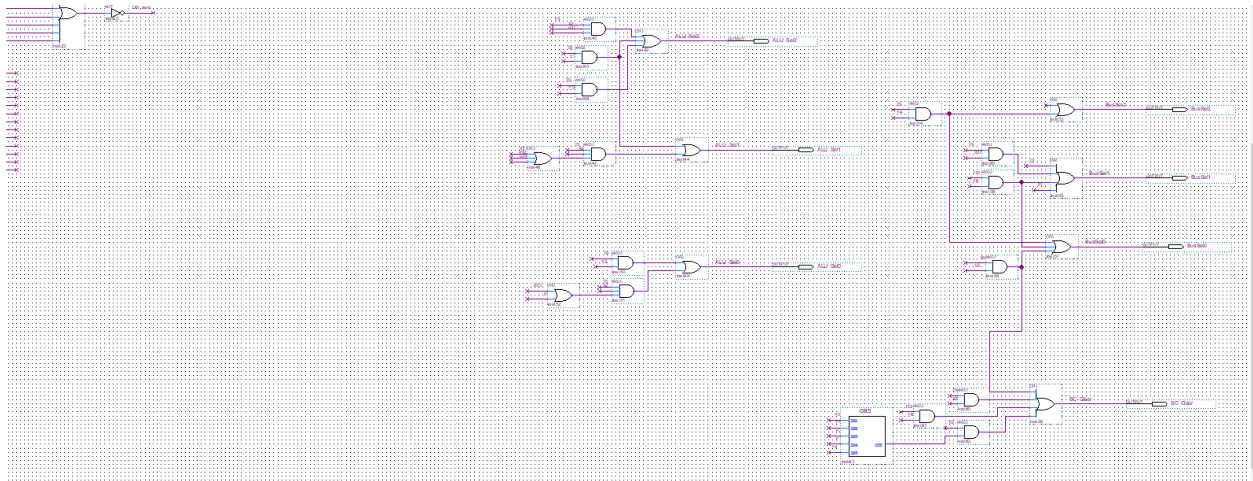


Figure 5: Second Half of Completed Circuit Diagram of lab4top

Discussion of Components

In the hardware part of the lab, we used basic logic gates (AND, OR, NOT) to implement the control signals. Multiplexers were used to choose values for the bus, and registers like AC, IR, DR, PC, and AR were already included in the basic computer design. Each control signal told these components when to load, clear, increment, or perform an ALU operation.

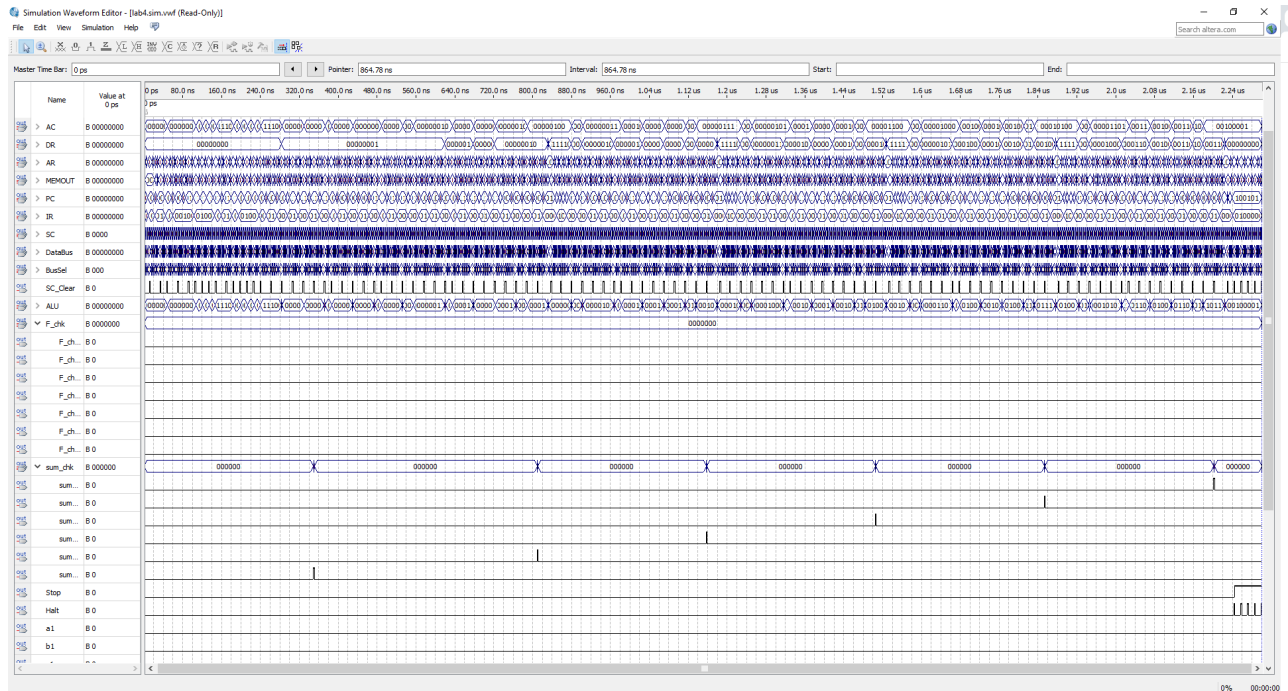
The ALU handled arithmetic and logic tasks, like incrementing AC or DR. The sequence counter (SC) and timing signals (T0–T12) controlled the timing of all micro-operations automatically.

Discussion of Solution

The solution followed the same steps described in the lab document. First, we identified every control signal and wrote the Boolean equations for when each signal should be active based on the instruction bits and the timing steps. Then, we used these equations to determine when each register should load, increment, or clear while the program was executing. After organizing all of the control signals, we checked that each signal activated at the correct time in the instruction cycle (T0–T12).

Simulation and Verification

Simulation Results



Verification

The CPU design was simulated using the provided Test1.vwf waveform file. During the simulation, we verified the correctness of our CPU design using the built-in checker signals. The sum_chk signal was correctly triggered, confirming that the sum of Fibonacci elements was calculated and stored accurately. However, the F_chk signal for the Fibonacci array was not triggered during our tests. We reviewed our hardware equations and schematic design multiple times with the TA and confirmed that the equations and circuit implementation were correct. The TA indicated that this issue was acceptable for the current stage of the lab and advised making a note of it in the report.

Discussion and Conclusion

Problems and Issues

During the lab, we encountered a few challenges. The checker signal F_chk for the Fibonacci array did not trigger, despite the correct implementation of control signals and verified circuit design; this may have been due to simulation limitations or minor synchronization timing issues in the waveform test. Deriving the Boolean expressions

for all control unit signals was also complex, requiring careful analysis of instruction cycles and timing signals, which was time-consuming and prone to minor errors. Additionally, multiple iterations of simulation and debugging were necessary to ensure that all signals were activated at the correct times, involving repeated consultation of tables and timing diagrams. All issues were addressed by reviewing and verifying the hardware equations and schematic design with the TA, confirming that the logic and overall CPU design were implemented correctly.

Conclusion

In this lab, we successfully designed and implemented the Control Unit for a simplified RISC CPU using Quartus II. We derived control signals from instruction bits and timing, implemented them with combinational logic and multiplexers, and verified correct CPU operation using simulation and checker signals. Minor issues, such as the F_chk signal not triggering, were noted and reviewed with the TA, confirming the overall design correctness. This lab highlighted the importance of systematic control signal derivation, careful timing analysis, and thorough verification to ensure a fully functioning CPU Control Unit.